

面向级联式语音对话系统打断一致性的 播放感知 KV 缓存管理

第一作者
单位院系
单位名称
城市, 国家
邮箱

第二作者
单位院系
单位名称
城市, 国家
邮箱

Abstract—级联式流水线（流式 ASR → LLM → 流式 TTS）仍是语音对话系统的主流架构，但在用户打断（**barge-in**）场景下存在结构性的一致性缺陷：LLM 的生成进度、TTS 的合成进度与音频实际播放进度三者不同步。若系统将已生成的全部内容记入对话历史，模型会在后续轮次引用用户从未听到的内容。本文形式化了“生成—合成—播放”三进度指针与听到边界语义，并在其上构建三项机制：（1）推测生成调度——以话轮检测模型的连续置信度驱动，将端点判断软化为可作废的过程；（2）播放感知的 KV 缓存管理——将播放位置反向映射到 token 边界，执行显式 KV 截断（注意力掩码与位置编码同步修正）并重建对话模板角色边界；（3）被打断历史的三种处理策略。在 7B 主模型与 MultiWOZ 派生数据上，朴素的生成位置截断使未听内容在至多 51% 的被打断轮次中复现（表面重叠口径；最保守的 LLM 裁判口径下为 2.7%），而本文的播放感知截断将该比率构造性归零；打断响应关键路径恒为亚毫秒且与上下文长度无关，KV 复用相对重新预填充在 8k 上下文处最高加速 39.7×；推测调度给出连续可调的浪费—延迟前沿，以约 4.5% 的 token 浪费将说完后首 token 延迟从 48.5 ms 降至 12.1 ms。据我们所知，这是首个在 KV 缓存层面实现播放感知上下文管理的开源、可复现级联实现。

Index Terms—语音对话系统，打断，KV 缓存，推测生成，上下文一致性，低延迟

I. 引言

大语言模型（LLM）使语音助手从指令式交互走向自然对话。工程实践中，级联架构——流式自动语音识别（ASR）、LLM 与流式语音合成（TTS）——凭借模块可控性与强推理能力仍是主流方案。然而在首 token 延迟之外，自然对话还要求从容处理打断：用户随时可能插话，系统能否连贯地继续，取决于一个隐蔽的问题——系统认为自己说了什么，与用户实际听到了什么，是否一致？

在级联架构中这一失配是结构性的。LLM 生成 token 快于 TTS 合成，合成又快于实时播放。当用户在播放中途打断时，把“已生成的全部内容”记入对话历史的系统，会在后续轮次引用用户从未听到的信息（“正如我刚才提到的……”），导致对话错乱。本文实验表明，在这种朴素截断下，未听内容引用在表面重叠口径下最高发生于 51% 的被打断轮次（第 V 节）。

“对话历史应等于用户实际听到的内容”这一原则已在商用系统中得到实践——OpenAI Realtime API [1]、微软 Azure Voice Live [2]、LiveKit Agents [3]——本文明确不主张该原则本身的原创性。但这些实现均为闭源，或仅在受管理的转写文本层面操作；据我们所知，尚无学术或开源工作在 LLM 推理内部以显式 KV 缓存操作实现播放感知的上下文管理。本文填补这一空白，贡献如下：

- 推测生成调度。以连续话轮检测置信度配推测阈值，将端点判断变为可作废、可回滚的过程；真实数据上的九点浪费—延迟前沿存在明显拐点：以约 4.5% 的 token 浪费将说完后首 token 延迟从 48.5 ms 降至 12.1 ms（第 IV-A、V 节）。
- 播放感知的 KV 缓存管理（核心）。四向映射时间轴（token ↔ 片段 ↔ 音频块 ↔ 采样区间）、经 DynamicCache.crop 的显式截断（掩码/位置同步处理）、对话模板角色边界重建与推测回滚。打断关键路径实测亚毫秒、与上下文长度无关；KV 复用在 8k 上下文处相对重新预填充最高加速 39.7×（第 IV-B、V 节）。
- 被打断历史的处理策略。朴素截断、打断标记与轻量模型重写的实现与消融（第 IV-C、V 节）。
- 开源、可复现的参考实现，附含人工仲裁的双口径一致性评测协议。

贡献 2 的高层理念是商用系统的 prior art；本文的新颖性在于开放、可复现、机制级的实现及其量化评测。软触发与重写器使用现成模型，不构成模型层贡献。

II. 相关工作

1) 商用实践：OpenAI Realtime API 提供 `conversation.item.truncate`：打断时客户端上报 `audio_end_ms`，服务端删除未播放音频及其转写，明确“使上下文不包含用户尚未听到的文本” [1]。Azure Voice Live 提供 `auto_truncate`，将会话上下文更新为已播放部分 [2]。LiveKit Agents 仅提交实际转发的转写并标记该消息 `interrupted` [3]。可见播放感知截断已是成熟工程实践；与本文的差异在于开源与闭源、显式 KV 操作与转写删除、实测播放与估计——而非该能力是否存在。

2) 学术流式对话：LTS-VoiceAgent [4] 以语义触发与增量推理降低“边听边想”延迟，但不涉及打断、播放位置或 KV 截断。RelayS2S [5] 用快速全双工模型推测起草回复前缀、慢速级联验证接力，与本文共享“以推测换延迟”的思路，但目标是起始延迟而非打断作废与 KV 管理。Moshi [6] 在帧同步端到端架构中将双方语音建模为并行流，“生成即播放”隐式成立——从反面印证三指针失配是级联架构特有的问题。FireRedChat [7] 以个性化流式 VAD 改进打断检测，但不管理 LLM 上下文中说了一半的回复。

3) KV 缓存操作：HuggingFace DynamicCache 提供 `crop` 原语 [8]；推理框架围绕前缀缓存做吞吐优化。正

交的 Agent 侧工作由文本或工具逻辑驱动 KV 剪枝或丢弃——IntentKV [9]、推测性工具调用 [10]——而非由音频播放进度驱动。据我们所知，尚无公开的开源先例在语音级联中按播放位置 crop KV 并重建对话模板角色边界。

III. 问题形式化

A. 流水线模型与进度指针

考虑级联 $\mathcal{S} = \langle \text{ASR}, \text{LLM}, \text{CHK}, \text{TTS}, \text{PLY} \rangle$ 。流式 ASR 产出确定性（不再修正的）文本片段 $u_1, u_2, \dots$ ，增量预填充 LLM 的 KV 缓存 $\mathcal{K}$ 。回复以自回归方式生成 token $y_1, \dots, y_G$ ；断句器 CHK 将 token 流切分为片段 f_j ，其 token 区间 $[\text{ts}(f_j), \text{te}(f_j))$ 连续衔接，是截断操作的原子单位。流式 TTS 将每个片段渲染为音频块，占据采样区间 $[\text{ss}(f_j), \text{se}(f_j))$ ；播放器回报已播采样数 p 。

任一时刻 t ，三个指针度量同一回复的进度：生成 $g(t)$ (token)、合成 $s(t)$ (片段)、播放 $p(t)$ (采样)。生成快于合成、合成快于实时播放，故

$$p(t) \leq s(t) \leq g(t) \quad \forall t. \quad (1)$$

图 1 展示了该失配。Moshi 等帧同步端到端模型使三指针合一；级联架构必须显式管理这一差距。

B. 听到边界与一致性

设打断发生时 p 落于片段 f_k 内 ($\text{ss}(f_k) < p \leq \text{se}(f_k)$)。听到边界为

$$H(p) = \text{te}(f_k), \quad (2)$$

即被打断片段整体向上取整计入“已听到”。历史满足用户感知一致性，当且仅当被打断轮的 assistant 内容恰保留在 $[0, H(p))$ 上。朴素做法保留全部 G 个 token，其中 $y_{H(p)+1..G}$ 为未听内容。量化误差 $\varepsilon(p) = H(p) - p^*$ (p^* 为 p 在 f_k 内按播放比例对应的 token 位置) 有界于单个片段；本文不隐藏该误差，而以下述 *strict* 口径显式量化。两类截断事件共享同一机制：播放期打断 ($p > 0$) 与推测作废 ($p = 0$ ，回滚到 $H = 0$)。

C. 反向映射与截断合法性

反向映射 $\Phi: p \rightarrow f_k \rightarrow [\text{ts}(f_k), \text{te}(f_k)) \rightarrow H(p)$ 须以近常数时间完成——它位于“打断即停”的关键路径上。将 $\mathcal{K}$ 截断至绝对位置 $N = a_0 + H(p)$ (a_0 为本轮 assistant 起点) 是合法的，当且仅当：(i) KV 张量、注意力掩码与记录的序列长度在 N 处一致；(ii) 后续位置编码自 N 连续；(iii) 对话模板角色边界被重建，使后续轮次结构合法。条件 (ii)(iii) 是主要工程陷阱；三项不变式在全部实验中做运行时检查。

D. 评测指标

延迟：TTFT_{eff} 度量“用户真实说完 $\rightarrow$ 首个可用 token” (用户体感口径；推测存活时 ≈ 0)；mouth-to-ear 为建模值——首片段就绪时刻加上实测设备画像的 TTS 首块延迟。一致性：未听内容引用率，分 *loose* 口径 (完整未听片段；本文方法下构造性为零——实验量化的是基线的失败率而非本文方法) 与 *strict* 口径 (另计半截片段的未播尾部，即 ε 的后果)。引用判定用两把一致性需报告的尺子：词面规则检测器 (系统性偏高；上界) 与特定信息引

用的 LLM 裁判 (保守；下界)，另以分层抽样的人工仲裁校验。效率：推测浪费率

$$\rho = \frac{\sum \text{被作废的推测 token 数}}{\sum \text{被作废的推测 token 数} + \sum \text{最终生成 token 数}}, \quad (3)$$

及阈值扫描下的 $(\rho, \text{TTFT}_{\text{eff}})$ 前沿。

IV. 系统设计

图 2 为总体架构：在一期“流式 ASR + 增量预填充”流水线之上，扩展输出侧三级流水 (断句 $\rightarrow$ 流式 TTS $\rightarrow$ 播放) 与两个控制机制。设计原则：(P1) 历史 = 用户所听 (原则本身为 prior art；本文贡献其在开源级联上的显式 KV 级实现)；(P2) 以冗余、可测的计算换交互流畅度。

A. 推测生成调度

现成话轮检测模型 (TEN [11]) 对累积的用户文本给出连续置信度 c_i 。当 $c_i \geq \theta$ 时，系统记录推测起点 $b := |\mathcal{K}|$ ，注入 assistant 角色标记，并预生成至多 B 个 token (推测预算限制单次作废成本)。若又有用户片段到达，推测作废：KV 截断回 b (播放截断在 $p = 0$ 时的退化情形)，产出计入 ρ 。若用户确已说完且推测存活，缓存 token 立即输出 ($\text{TTFT}_{\text{eff}} \approx 0$) 并继续生成。触发器在文本侧与该片段的 KV 预填充并行运行，不增加端到端延迟。扫描 θ 把端点判断变为 $(\rho, \text{TTFT}_{\text{eff}})$ 前沿上的可调旋钮。

B. 播放感知的 KV 缓存管理

1) 四向映射时间轴：以片段记录为主轴的追加式结构，每条记录为 $\langle f_j, [\text{ts}, \text{te}], \text{音频块列表}, [\text{ss}, \text{se}], \text{状态} \rangle$ ，状态机为：推测 $\rightarrow$ 合成中 $\rightarrow$ 已入队 $\rightarrow$ 播放中 $\rightarrow$ 已播完/已作废。三个生产者各写自己的维度 (生成线程：token $\leftrightarrow$ 片段；TTS 线程：片段 $\leftrightarrow$ 音频块与采样轴；播放线程：原子更新 p)。每轮片段数为小量，全表单锁加无锁播放游标即满足亚毫秒反查需求。token 区间来自断句文本与解码 token 流之间的非空白字符计数对齐，经验证无损。

2) 合法截断：打断时， $\Phi(p)$ 给出 $H(p)$ ，缓存截断至 N ：KV 张量切片 (DynamicCache.crop [8])、注意力掩码同步截短至 N (掩码长度是后续 past length 的事实来源——只截 KV 不截掩码会静默破坏位置编码)、账本更新使后续预填充自 N 显式重算位置。关键路径就是这次张量切片：亚毫秒。

3) 角色边界重建：KV 是扁平序列；角色结构只存在于对话模板标记中。我们通过差分从 tokenizer 模板推导角色切换串 (如 ChatML 的 $\langle |\text{im_end}| \dots \langle |\text{im_start}| \rangle \text{user} \rangle$)，并将其作为普通文本经同一显式掩码/位置路径预填充——无需特殊算子。该重建 (数十毫秒) 不在关键路径上：停播加截断已足够形成体感”立即停止”，重建可推迟到下一轮用户输入前。生成循环把每个新 token 的 KV、掩码与 id 写回同一累积缓存，使 assistant 侧 KV 成为跨多轮的一等可截断状态。

C. 被打断历史的处理策略

播放截断后历史可能停在半句。我们在其上比较：朴素 (不处理)；标记 (追加打断标记，零成本)；重写 (仅对半截情形，0.6B 模型把半句改写为自然收尾、约束不新增信息；被打断 assistant 段的 KV 按重写文本重算，重写与用户的打断语音并行执行)。

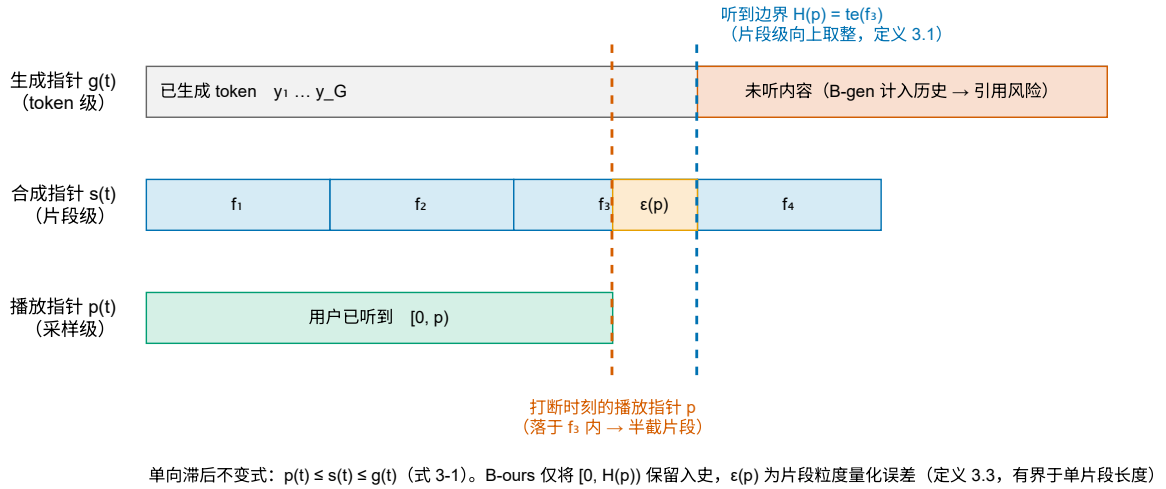


Fig. 1. 三进度指针与听到边界。打断时播放指针 p 落于片段 f_3 内；听到边界 $H(p) = te(f_3)$ 按片段粒度向上取整， $\epsilon(p)$ 为有界的量化误差。 p 与 g 之间为未听内容；将其保留在历史中（B-gen）构成引用风险。

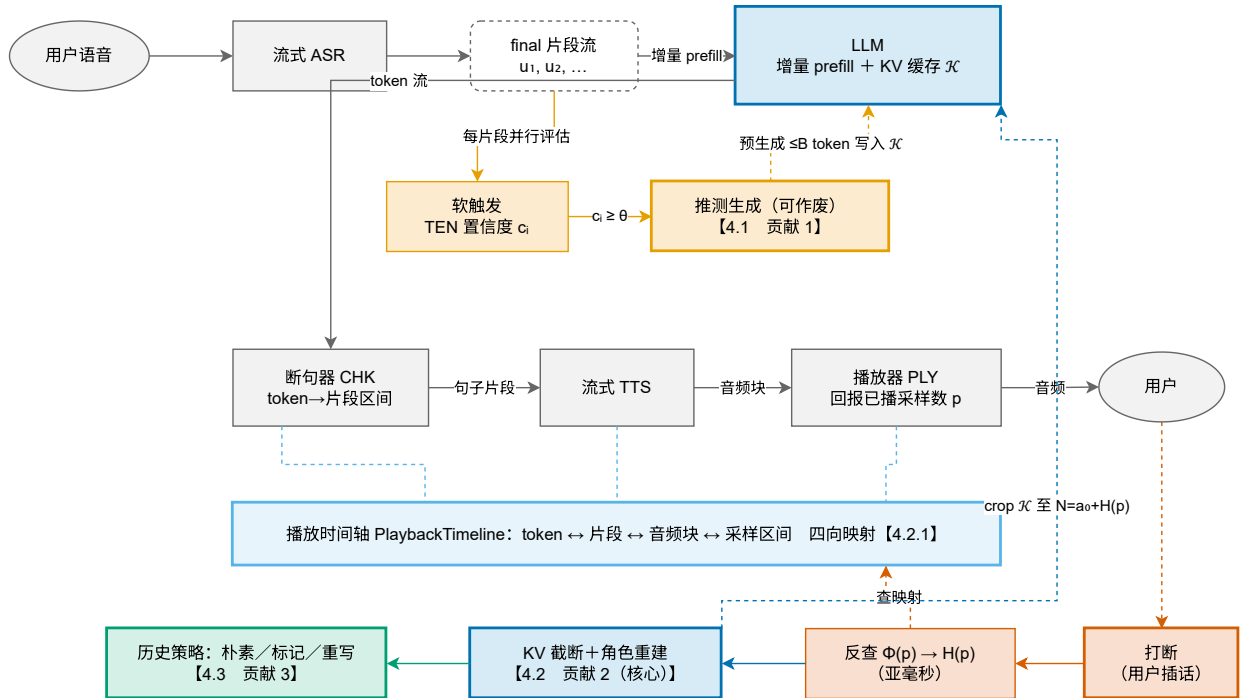


Fig. 2. 系统架构。上：输入流水线与软触发驱动的推测生成（第 IV-A 节）。中：输出流水线与四向映射时间轴。下：打断链路——反查 $\Phi(p) \rightarrow H(p)$ 、KV 截断 + 角色重建（第 IV-B 节）、历史策略（第 IV-C 节）。

V. 实验评估

A. 实验设置

两块 RTX 3090；主 LLM Qwen2-7B-Instruct [12]；软触发 TEN Turn Detection (7.6B, 文本侧；在本文话语上标定成对 AUC = 1.00) [11]；重写器 Qwen3-0.6B [13]；LLM 裁判 Mistral-7B-Instruct-v0.3 [14]（与主模型不同家族，避免同族偏袒）；流式 TTS CosyVoice2-0.5B [15]，时序画像实测（24kHz 下 3175 采样/非空白字符；3090 无 TensorRT 首块延迟 2433.6ms；RTF 0.513）。数据自

MultiWOZ 2.1 [16] 派生（固定种子）：103 条多轮对话与 100 条子句切分话语，英文。打断以确定性程序注入于播放比例 25%/50%/75% 及干净片段边界（对照组），以播放指针为真值——完全可复现。被测条件：System A（非流式基线：说完后全量预填充、生成完毕、整段合成）、B-ours（播放截断）、B-gen（生成截断，保留全部已生成内容）。合成位置截断在本文同步合成模拟下与 B-gen 等价，不作为已验证条件。

TABLE I
未听内容引用率 (E3, 每条件 $n=412$)

口径	检测器	B-ours	B-gen	p
loose	规则	0.0% (构造性)	51.0%	1.8×10^{-79}
strict	规则	51.0%	73.3%	5.0×10^{-11}
loose	裁判	0.0%	2.7%	9.1×10^{-4}
strict	裁判	2.4%	2.9%	0.83 (不显著)

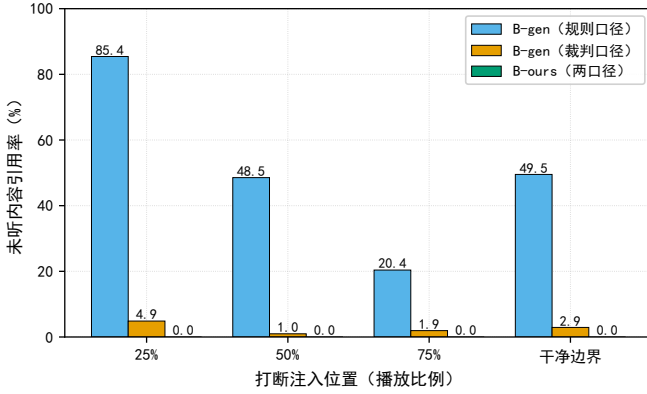


Fig. 3. 分注入位置的未听内容引用率。B-gen 随打断位置后移单调下降；B-ours 两口径均为零——机制的构造性保证，而非统计结果。

B. 一致性 (主结果)

表 I 报告未听内容引用率 (每条件 $n=412$)。loose 口径下 B-ours 构造性为零 (机制使历史中不存在未听片段)——实测量化的是基线的失败率：B-gen 的后续回复中 51.0% 复用了未听的表面内容，即便按最保守的裁判口径也有 2.7% 的轮次让模型“引用了自己从未说出口的话” (Fisher 精确检验 $p < 10^{-3}$)。打断越早比率越高 (规则口径：25% 播放处 85.4% vs. 75% 处 20.4%；图 3)。strict 口径下，半截片段的残余尾部使规则检测器对 B-ours 报 51.0%——但这是词面重叠上界；裁判口径报 2.4%，与 B-gen 的 2.9% 无统计差异 ($p=0.83$)：片段粒度截断在“特定信息引用”意义上没有可测量的额外代价。干净边界注入组 103/103 样本 strict 残余为零，为边界语义提供数据级校验。

两把尺子在真实数据上分歧很大 (Cohen’s $\kappa \approx 0.05$)：MultiWOZ 领域词 (hotel、book、train) 使规则检测器过触发。因此本文将规则口径读作上界、裁判读作下界，并以 37 条人工标注仲裁：人一裁判一致性中等偏强 (84.6%， $\kappa = 0.649$)，人一规则一致性不优于随机 ($\kappa = -0.073$)，支持以裁判为主口径。尤其重要的是，strict 口径 24 条样本人工判定 0/24 构成真实特定引用，为片段级截断粒度提供了实证辩护。

C. 浪费—延迟前沿

图 4 为 θ 的九点扫描 (每点 $n=100$)，含“永不推测”哨兵。前沿单调且有清晰拐点 ($\theta \in [0.85, 0.97]$)： $\theta=0.92$ 时 4.5% 的浪费换得 TTF_{eff} 48.5→12.1 ms (−75%)； $\theta=0.9688$ 时 0.8% 的浪费仍有 −40%。激进端 ($\theta \leq 0.4$) 浪费 14–29% 而 $\text{TTF}_{\text{eff}} \approx 0$ ：推测几乎总在用户说完前就绪。注意 7B 下“永不推测”的全额 TTF 因流式预填

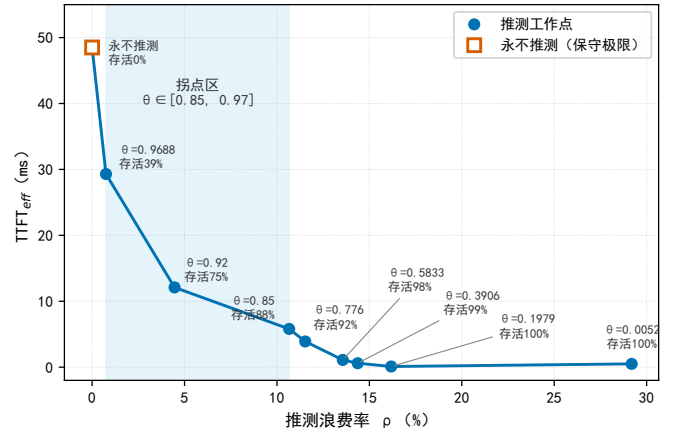


Fig. 4. 推测浪费率与 TTF_{eff} ：连续可调的前沿；空心方块为“永不推测”保守极限。

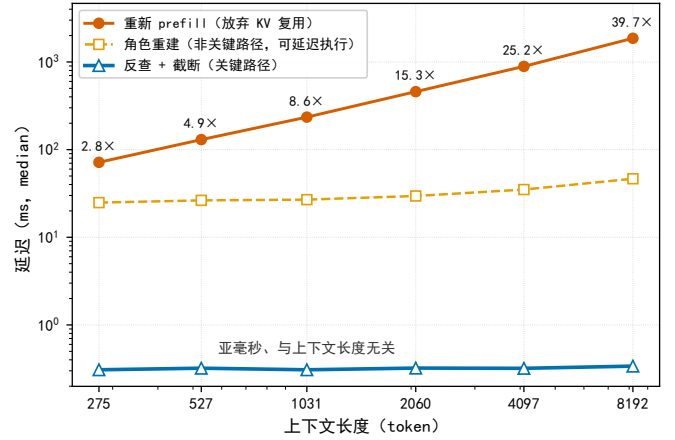


Fig. 5. 打断响应路径与重新预填充随上下文长度的延迟 (双对数)。反查+截断亚毫秒且与长度无关；重新预填充近线性增长 (8k 处 39.7×)。

充仅 48.5 ms，故相对收益 (75–99%) 比绝对毫秒数更具外推意义；模型更大或回复前缀更长时绝对收益同步放大。

D. 端到端延迟

在 $\theta=0.391$ 工作点 ($n=100$)，实测 TTF：System A 27.4 ms vs. B-ours TTF_{eff} **0.6 ms** (降低 97.9%)。建模 mouth-to-ear (首片段就绪 + TTS 首块, 3090 画像)：A 9080 ms (须等全部生成再整段合成) vs. B-ours 2482 ms，其中 2434 ms 为 TTS 首块延迟。该项强依赖部署条件：按厂商报告的 A100+TensorRT 首块指标 (≈ 45 ms) 外推，B-ours 可至 ~ 100 ms 量级，而 System A 的“等全部完成”是随回复长度增长的结构成本。

E. KV 复用与重新预填充

图 5 扫描上下文长度 275–8192 token (5 次取中位数, 7B)。关键路径 (反查+截断) 为 0.31–0.34 ms、与上下文长度无关；角色重建 (25–47 ms) 不在关键路径；对听到边界前上下文重新预填充随长度近线性增长，8k 处达 1.86 s——39.7× 的差距随多轮上下文增长而扩大。

F. 被打断历史策略（诚实的零结果）

裁判连贯性评分（1–5，每策略 $n=100$ ）：朴素 3.76、重写 3.62、标记 3.29；重写未优于朴素截断（Wilcoxon $p=0.32$ ）。可能原因：7B 主模型对半句历史已足够鲁棒；0.6B 重写器的改写稀释了上下文线索。工程结论仍成立——重写延迟均值 639 ms、P90 937 ms，可隐藏于用户打断说话期内——故本文将重写定位为探索性机制，留待更强重写器与成对偏好评测检验。

VI. 讨论与局限

与商用系统的关系。差异在于操作层次（显式 KV 截断保留跨轮 KV 复用 vs. 转写删除）、位置来源（播放器回报采样数并反向映射 vs. 实时速率假设或客户端上报偏移）与可检视性（全部映射与决策落盘；实验可复现）。这些差异均不支持“商用系统缺乏该能力”的论断。测量仪器。规则检测器在领域词上过触发；上界/下界/仲裁协议可缓解，但更大规模人工标注能收紧区间。时序模型。mouth-to-ear 为画像建模值；首块延迟跨推理栈相差两个数量级，跨系统比较需谨慎。范围。实验限于英文任务型对话与确定性注入打断；开放域语音、自然打断行为与中文路径留待后续。机制本身不绑定特定栈：任何句子级流式 TTS、DynamicCache 生态中的任何因果 LM、任何输出连续置信度的端点模型均可替换。

VII. 结论

本文形式化了使级联语音代理“记错自己说过什么”的生成—合成—播放失配，并给出一个开放可复现的级联实现，使对话历史可证明地等于用户所听：推测调度把端点检测变为可调的浪费—延迟旋钮（4.5% 浪费下 $48.5 \rightarrow 12.1$ ms）；播放感知 KV 截断以与上下文长度无关的亚毫秒时间响应打断（8k 处相对重新预填充 $39.7\times$ ）；含人工仲裁的双口径协议表明朴素截断使未听内容在 51%/2.7%（表面/特定）的被打断轮次复现，而本文机制将其构造性消除，且片段粒度截断无可测代价。代码、数据派生与逐轮埋点均开源以供复现。

致谢

本文写作过程中使用大语言模型辅助文献检索核验、绘图脚本与语言润色；全部技术内容、实验与结论由作者完成并核验。

REFERENCES

- [1] OpenAI, “Realtime API — conversations,” <https://developers.openai.com/api/docs/guides/realtime-conversations>, 2025, accessed: 2026-07-21.
- [2] Microsoft Azure, “Voice Live auto-truncation,” <https://learn.microsoft.com/en-us/azure/ai-services/speech-service/how-to-voice-live-auto-truncation>, 2025, accessed: 2026-07-21.
- [3] LiveKit, “Agents — events and interruption handling,” <https://docs.livekit.io/agents/build/events/>, 2025, accessed: 2026-07-21.
- [4] W. Zou, Y. Miao, Z. Ma, J. Xu *et al.*, “LTS-VoiceAgent: A listen-think-speak framework for efficient streaming voice interaction via semantic triggering and incremental reasoning,” 2026, arXiv:2601.19952.
- [5] L. Mai, “RelayS2S: A dual-path speculative generation for real-time dialogue,” 2026, arXiv:2603.23346.
- [6] A. Défossez, L. Mazaré, M. Orsini, A. Royer, P. Pérez, H. Jégou, E. Grave, and N. Zeghidour, “Moshi: A speech-text foundation model for real-time dialogue,” 2024, arXiv:2410.00037.
- [7] FireRed Team, “FireRedChat: Streaming personalized VAD for barge-in,” 2025, arXiv:2509.06502.
- [8] Hugging Face, “Transformers documentation: KV cache strategies,” https://huggingface.co/docs/transformers/en/kv_cache, 2025, accessed: 2026-07-21.
- [9] J. Li, J. Lou, and J. Li, “IntentKV: Cross-turn intent-aware KV cache pruning for agent inference,” 2026, arXiv:2606.09916.
- [10] C. Hooper, M. Kang, S. Moon *et al.*, “Speculative interaction agents: Building real-time agents with asynchronous I/O and speculative tool calling,” 2026, arXiv:2605.13360.
- [11] TEN Framework, “TEN turn detection: Turn detection for full-duplex dialogue communication,” <https://github.com/TEN-framework/ten-turn-detection>, 2025, accessed: 2026-07-21.
- [12] A. Yang *et al.*, “Qwen2 technical report,” 2024, arXiv:2407.10671.
- [13] —, “Qwen3 technical report,” 2025, arXiv:2505.09388.
- [14] A. Q. Jiang, A. Sablayrolles, A. Mensch *et al.*, “Mistral 7B,” 2023, arXiv:2310.06825.
- [15] Z. Du *et al.*, “CosyVoice 2: Scalable streaming speech synthesis with large language models,” 2024, arXiv:2412.10117.
- [16] M. Eric, R. Goel, S. Paul, A. Sethi, S. Agarwal, S. Gao, A. Kumar, A. Goyal, P. Ku, and D. Hakkani-Tür, “MultiWOZ 2.1: A consolidated multi-domain dialogue dataset with state corrections and state tracking baselines,” in *Proc. 12th Language Resources and Evaluation Conf. (LREC)*, 2020, pp. 422–428.